

Day 5 — 并发编程入门（完整版）

一、理解并发：为什么 Python 的并发不一样

1. 是什么

并发（Concurrency）是让程序“同时”做多件事的能力。但 Python 里有两个陷阱：

概念	一句话	能不能加速 CPU 密集任务
多线程 (threading)	多个执行流交替跑，共用内存	✗ 因为 GIL
多进程 (multiprocessing)	多个解释器隔离跑，各自独立	✓ 真并行
异步 I/O (asyncio)	一个线程里切换任务，等 I/O 时干别的	✗ 单线程
并发 (concurrent.futures)	上面三者的高级封装	看底层选谁

关键前提：**GIL**（全局解释器锁）

CPython 解释器设计上的一个限制——同一时刻只有一个线程在执行 Python 字节码。

```
import threading, time

def count(n):
    while n > 0:
        n -= 1

# 单线程
start = time.time()
```

```

count(50_000_000)
print(f"单线程: {time.time() - start:.2f}s")

# 两线程 — 比单线程还慢（加锁开销）
t1 = threading.Thread(target=count, args=(25_000_000,))
t2 = threading.Thread(target=count, args=(25_000_000,))
start = time.time()
t1.start(); t2.start()
t1.join(); t2.join()
print(f"两线程: {time.time() - start:.2f}s")

```

2. 解决了什么问题

选错了并发方案，性能反而下降。

任务类型	举例	正确方案	错误方案
CPU 密集	计算、排序、图像处理	multiprocessing	threading ✕
I/O 密集	网络请求、文件读写、数据库	asyncio / threading	—
混合型	计算+等待交替	组合方案	—

```

# ✕ 错误：CPU 密集用 threading
def cpu_heavy():
    total = 0
    for i in range(10_000_000):
        total += i ** 2
    return total

t1 = threading.Thread(target=cpu_heavy) # GIL 锁死，白忙活

# ✔ 正确：CPU 密集用 multiprocessing
from multiprocessing import Pool
with Pool() as pool:
    results = pool.map(cpu_heavy, range(4))

```

3. 选择策略

你的场景	用这个
大量计算，要多核加速	multiprocessing / ProcessPoolExecutor
高并发网络请求/API 调用	asyncio + aiohttp
I/O 密集但代码已是同步的	ThreadPoolExecutor (简单改造)
需要和已有同步库配合	ThreadPoolExecutor
追求极致性能 + 新项目	asyncio
不想动脑子	concurrent.futures (统一接口)

二、threading — 多线程

1. 是什么

Python 标准库提供的线程编程接口。一个进程内的多个线程共享内存，适合 I/O 等待场景。

2. 解决了什么问题

不让程序在等待 I/O 时傻站着。

```
# ✔ 多线程：三个同时等
threads = [
    threading.Thread(target=download, args=(f"url{i}",))
    for i in range(3)
]
start = time.time()
for t in threads: t.start()
for t in threads: t.join()
print(f"多线程耗时: {time.time() - start:.1f}s") # ~2s
```

3. 核心理论

```
import threading, time

def worker(name, delay):
    print(f"{name} 启动")
    time.sleep(delay)
    print(f"{name} 完成")

# 创建并启动
threads = []
for i in range(3):
    t = threading.Thread(target=worker, args=(f"Worker-{i}", i))
    threads.append(t)
    t.start()

# 等待全部完成
for t in threads:
    t.join()

# 守护线程（主线程退出时自动结束）
t = threading.Thread(target=worker, args=("Daemon", 10), daemon=True)
t.start()
```

同步工具

同步工具	作用	场景
Lock	互斥锁，一次一个线程	保护共享数据
RLock	可重入锁，同一线程可多次 lock	递归函数中加锁
Semaphore	信号量，限制 N 个线程同时访问	限流、连接池
Event	事件标志，一个线程发信号 N 个等	等待条件满足
Condition	条件变量，更复杂的事件通知	生产者-消费者

```

# ✘ 不加锁：累加不是原子操作
counter = 0
def increment():
    global counter
    for _ in range(100_000):
        counter += 1 # 读→改→写，三条指令！

# ✔ 加锁
class SafeCounter:
    def __init__(self):
        self.value = 0
        self._lock = threading.Lock()

    def increment(self):
        with self._lock:
            self.value += 1

# RLock — 可重入
lock = threading.RLock()
def recursive(n):
    with lock:
        if n > 0:
            recursive(n - 1) # 不会死锁

# Event — 信号等待
ready = threading.Event()
def waiter():
    ready.wait() # 阻塞直到 set
    print("开冲！")

def setter():
    time.sleep(2)
    ready.set()

# Semaphore — 限制并发
sem = threading.Semaphore(3)
def limited():
    with sem:
        print(f"允许并发")

```

4. 生产者-消费者模式

```
import queue

q = queue.Queue(maxsize=10)

def producer():
    for i in range(5):
        q.put(f"item-{i}")
        print(f"生产: item-{i}")
    q.put(None)  # 哨兵信号

def consumer():
    while True:
        item = q.get()
        if item is None:
            break
        print(f"消费: {item}")
    q.task_done()
```

5. ThreadPoolExecutor — 高级封装

```
from concurrent.futures import ThreadPoolExecutor, as_completed

def fetch_url(url):
    time.sleep(1)
    return f"数据来自 {url}"

urls = [f"http://api.example.com/{i}" for i in range(10)]

with ThreadPoolExecutor(max_workers=5) as executor:
    # submit — 逐个提交
    futures = {executor.submit(fetch_url, url): url for url in urls}
    for f in as_completed(futures):
        print(f.result())

    # map — 批量提交（保持顺序）
    results = list(executor.map(fetch_url, urls))
```

三、multiprocessing — 多进程

1. 是什么

绕过 GIL 的唯一方式——启动多个 Python 进程，每个有自己独立的解释器和内存空间。

2. 解决了什么问题

CPU 密集任务真正用上多核。

3. 核心理论

```
from multiprocessing import Process

def cpu_heavy(n):
    total = 0
    for i in range(n):
        total += i ** 2
    return total

p = Process(target=cpu_heavy, args=(10_000_000,))
p.start()
p.join()
```

ProcessPoolExecutor

```
from multiprocessing import Pool, cpu_count
from concurrent.futures import ProcessPoolExecutor

# Pool (经典 API)
with Pool(processes=cpu_count()) as pool:
    results = pool.map(cpu_heavy, [10_000_000] * 4)

# ProcessPoolExecutor (统一接口, 推荐)
```

```
with ProcessPoolExecutor(max_workers=cpu_count()) as executor:
    results = list(executor.map(cpu_heavy, [10_000_000] * 4))
```

进程间通信

通信方式	速度	复杂度	场景
Queue	中等	低	生产者-消费者
Pipe	快	中	双向通信
Value / Array	快	低	简单共享数据
shared_memory	最快	高	大块数据共享

```
from multiprocessing import Queue, Pipe, Value, Array

# Queue
q = Queue()
q.put("data")
item = q.get()

# Pipe
parent_conn, child_conn = Pipe()
parent_conn.send(42)
child_conn.recv() # 42

# 共享内存
counter = Value("i", 0) # int
counter.value += 1
arr = Array("d", [1.0, 2.0, 3.0]) # double[]
```

⚠ multiprocessing 的坑：

1. Windows 上需要 `if __name__ == "__main__":` 保护
2. 进程间不能共享普通 Python 对象（需要序列化）
3. 启动开销大（每个进程要 import 全部模块）
4. 调试困难（多进程，日志/异常可能丢失）

5. 资源隔离：一个进程 crash 不影响其他进程

四、asyncio — 异步 I/O

1. 是什么

用一个线程、一个事件循环，在等待 I/O 时自动切换到其他任务。不是并行，是并发。

时间轴 →

线程：[task1] 等待I/O [task2] 等待I/O [task1]...

↓ 切换 ↓ 切换

2. 解决了什么问题

海量 I/O 密集任务的高效方案。

3. 核心概念

概念	一句话	类似 threading 的什么
<code>async def</code>	定义一个协程函数	类似 <code>Thread(target=...)</code>
<code>await</code>	交出控制权，等结果回来再继续	类似 <code>t.join()</code>
<code>asyncio.run()</code>	启动事件循环	类似 <code>t.start()</code>
<code>asyncio.create_task()</code>	把协程注册到事件循环	类似创建线程
<code>asyncio.gather()</code>	并发跑多个协程	类似 <code>t.join()</code> 多个

4. 核心理论

```
import asyncio

async def fetch_data(url):
    print(f"请求 {url}")
    await asyncio.sleep(1)  # 模拟 I/O 等待
    print(f"完成 {url}")
    return {"url": url, "data": "..."}

# 方式1：单个协程
result = asyncio.run(fetch_data("http://example.com"))

# 方式2：并发执行
async def main():
    tasks = [
        asyncio.create_task(fetch_data(f"url-{i}"))
        for i in range(3)
    ]
    results = await asyncio.gather(*tasks)
    return results

results = asyncio.run(main())
```

规则

```
# ✗ await 只能在 async def 内
def wrong():
    await asyncio.sleep(1)  # SyntaxError!

# ✔ 正确
async def correct():
    await asyncio.sleep(1)
```

I/O 密集 ✓ vs CPU 密集 ✕

```
# ✓ I/O 密集 – 最佳场景
async def fetch_all():
    import aiohttp
    async with aiohttp.ClientSession() as session:
        tasks = [session.get(url) for url in urls]
        responses = await asyncio.gather(*tasks)
        return [await r.json() for r in responses]

# ✕ CPU 密集 – 会阻塞事件循环
async def compute():
    for i in range(10_000_000):
        pass # 一直不 await，事件循环卡死！
    return 42
```

混合方案：run_in_executor

```
import asyncio
from concurrent.futures import ThreadPoolExecutor

async def main():
    loop = asyncio.get_running_loop()
    with ThreadPoolExecutor() as pool:
        result = await loop.run_in_executor(
            pool,
            blocking_io_operation, # 同步函数
            arg1, arg2
        )
```

同步原语

```
sem = asyncio.Semaphore(3) # 限制并发数
lock = asyncio.Lock()      # 互斥
q = asyncio.Queue()        # 队列

async def fetch_with_limit():
```

```
    async with sem:
        await fetch_url(url)

# 超时控制
try:
    result = await asyncio.wait_for(
        fetch_url(url),
        timeout=5.0
    )
except asyncio.TimeoutError:
    print("请求超时")
```

5. 完整例子：并发下载器

```
import asyncio
import aiohttp

async def download_one(session, url):
    async with session.get(url) as resp:
        content = await resp.read()
        filename = url.split("/")[-1]
        with open(filename, "wb") as f:
            f.write(content)
        return filename

async def download_many(urls):
    async with aiohttp.ClientSession() as session:
        tasks = [download_one(session, url) for url in urls]
        return await asyncio.gather(*tasks)

urls = [
    "https://example.com/file1.jpg",
    "https://example.com/file2.jpg",
]

results = asyncio.run(download_many(urls))
```

今日练习

1. **ThreadPoolExecutor** 下载器：用 `ThreadPoolExecutor` 并发下载多张图片
2. **asyncio** 爬虫：用 `aiohttp` 并发请求 10 个 API，加超时 + 错误处理 + 重试
3. 多进程计算：用 `ProcessPoolExecutor` 并行计算多个大数的质因数分解
4. 综合练习：先用 `asyncio` 下载文件，再用 `ProcessPoolExecutor` 处理图片（缩略图生成）

明天预告：Day 6 — 工程化实战（`pytest`、`CLI` 工具、项目结构、`SQLAlchemy`、`FastAPI`）